

GUÍA DE TRABAJO — EJERCICIO 3

(Nutridas, Amistades y Colonias)

POO — composición, visibilidad, arrays, métodos privados, campos estáticos

1. Antes de programar: visualizar el modelo

Este ejercicio mezcla tres clases en un sistema realista.

Debes entender cómo se relacionan:

- Una **Amistad** depende de una Nutrida.
- Una **Nutrida** puede tener varias amistades.
- Una **Colonia** contiene muchas Nutridas.
- **GestorColonias** es una clase de utilidades (todo estático).

La clave es que entiendas **el orden correcto de creación** y la idea de **composición**.

Preguntas guía:

1. ¿Puede existir una amistad sin una nutrida objetivo?
2. ¿Puede existir una nutrida sin amistades?
3. ¿Qué estructura sería para almacenar amistades?
4. ¿Qué significa CAPACIDAD_MAXIMA en una colonia?
5. ¿Por qué algunos métodos son privados?

2. Clase Amistad — Cómo pensarla

Debes dividir la clase en 4 partes:

2.1. Atributos

- Una nutrida objetivo (**otra**).
- Una intensidad (1 a 10).

Debe entenderse como:

Una amistad siempre apunta a una nutrida concreta.

2.2. Campo estático

- **INTENSIDAD_MINIMA_ALTA = 7.**

Preguntas:

- ¿Qué significa que sea estático?
- ¿Se comparte entre todas las amistades?

2.3. Constructor con validación

El constructor recibe una intensidad.
El método privado decide si es válida.

Debes razonar:

- Si la intensidad no está entre 1 y 10 → asignar 1.
- ¿Por qué no puede estar el cuerpo de validación dentro del constructor directamente?
→ Buena razón para tener métodos privados.

2.4. Método público esIntensa

Debe devolver true/false según la intensidad.

3. Clase Nutrida — Cómo estructurarla mentalmente

Deberías poder ver **tres bloques**:

3.1. Atributos

- nombre
- edad
- peso
- array de amistades (hasta 5)

3.2. Constructores

- Uno **sin amistades**.
- Otro **con amistades**, que debe **copiar el array**, no asignarlo.

¿Por qué no lo asigno como el resto de campos?:

Si asigno directamente el array, cualquiera puede modificarlo desde fuera.
Copiarlo garantiza encapsulación.

3.3. Métodos importantes

- `esJuvenil()` → < 3 años
- `tieneAmistadCon()` → recorrer array
- `contarAmistadesIntensas()` → usar `esIntensa()`
- `contarAmistadesReales()` → método privado, solo contar posiciones no nulas.

Preguntas guía:

- ¿Qué ocurre si hay posiciones null?
- ¿Conviene validar antes de acceder al array?

4. Clase Colonia — Cómo planificarla

4.1. Atributos

- nombre
- array de nutridas

4.2. Campos estáticos

- `CAPACIDAD_MAXIMA = 20`
- `coloniasCreadas` (contador)

Preguntas:

- ¿Por qué `CAPACIDAD_MAXIMA`?
- ¿Qué sentido tiene el contador?

4.3. Constructores

- Uno recibe nutridas.
- Otro crea colonia vacía (array tamaño 0).

4.4. Métodos clave

1. `pesoMedio`

- recorrer nutridas
- sumar pesos
- dividir entre las nutridas reales

2. `buscarPorNombre`

- recorrer
- comparar nombres

- devolver nutrida o null

3. **contarEnlacesAmistad**

- recorrer cada nutrida
- recorrer sus amistades
- contar todas las posiciones no null

4. **contarNutridasReales** (privado)

- contar cuántas nutridas hay realmente (no nulls).
- Ayuda a que otros métodos calculen correctamente.

5. Clase **GestorColonias** — Cómo razonar métodos estáticos

Debe contener **lógica útil** que no “vive” dentro de una sola colonia.

5.1. **crearColoniaFiltrada**

Pasos que deben identificar:

1. Recibir colonia original.
2. Contar nutridas cuyo peso $\geq$ pesoMinimo.
3. Crear array exacto.
4. Copiar solo nutridas válidas.
5. Devolver nueva colonia.

Debes razonar cómo trabajar con arrays de tamaño preciso.

5.2. sonCompatibles

Comprobar:

- diferencia de edad ≤ 5
- peso conjunto ≤ 25
- amistades de cada nutrida < 5

Preguntas de razonamiento:

- ¿Cómo calcular la diferencia absoluta?
- ¿Cómo contar amistades reales de una nutrida?
- ¿Es correcto usar el método privado?
→ No, porque es privado; debes recurrir a observar posiciones no null.

6. Planificación del main

El main debe demostrar todas las clases.

Pero debes entender el **orden correcto** para crear objetos:

1. Crear nutridas sin amistades.
2. Crear amistades que apuntan a nutridas existentes.
3. Crear nutridas con amistades preasignadas.
4. Crear colonias.
5. Probar métodos de Colonias.
6. Probar métodos de GestorColonias.

Preguntas guía:

- ¿Por qué no puedo crear amistades antes que nutridas?
- ¿Cómo demuestro que la intensidad se valida automáticamente?
- ¿Cómo pruebo el método esIntensa?
- ¿Qué casos revelan errores de diseño?

7. Esquema visual para que entiendas el flujo general

Nutrdias independientes

↓

Amistades (dependen de nutridas ya creadas)

↓

Nutridas con amistades

↓

Colonias que agrupan nutridas

↓

GestorColonias analiza o genera nuevas colonias

8. Errores típicos a evitar

- No copiar el array de amistades en el constructor de Nutrida.
- Intentar usar el método privado desde fuera (no se puede).
- Olvidar comprobar nulls al contar amistades.
- Olvidar que los contadores estáticos deben incrementarse en los constructores.
- Confundir “amistad intensa” con “amistad total”.

9. Resumen de la guía para el alumno

1. Entiende las relaciones entre clases.
2. Respeta el orden lógico de creación de objetos.
3. Identifica qué métodos son privados y por qué.
4. Practica recorrer arrays parcialmente llenos.
5. Usa composición, no herencia.

6. Separa responsabilidades:

- Amistad: validar y evaluar
- Nutrida: información individual
- Colonia: lógica colectiva
- GestorColonias: utilidades

7. Monta el main como demostración, no como contenedor de lógica.

GUÍA PRÁCTICA — EJERCICIO 3

(Nutridas, Amistades, Colonias, GestorColonias)

1. CLASE Amistad

Atributos privados

Sugerencia de estructura:

```
private Nutrida /* nombre variable */;  
private int /* intensidad */;
```

Campo estático

```
public static final int INTENSIDAD_MINIMA_ALTA = /* valor */;
```

Constructor con validación

Plantilla incompleta:

```
public Amistad(Nutrida otra, int intensidad) {  
    this.otra = otra;  
  
    if (validarIntensidad(intensidad)) {  
        this.intensidad = intensidad;  
    } else {  
        // asignar valor mínimo permitido  
        this.intensidad = /* ??? */;  
    }  
}
```

Método esIntensa

```
public boolean esIntensa() {  
    return /* comparar intensidad con INTENSIDAD_MINIMA_ALTA */;  
}
```

Método privado validarIntensidad

Estructura sin completar:

```
private boolean validarIntensidad(int valor) {  
    return (valor >= /* ??? */ && valor <= /* ??? */);  
}
```

2. CLASE Nutrida

Atributos privados

Indica estructura:

```
private String nombre;  
private int edad;  
private double peso;  
private Amistad[] amistades;
```

Campo estático contador

La idea, no el cuerpo:

```
public static int totalNutridasCreadas = /* ??? */;
```

Constructor 1 (sin amistades)

Plantilla base:

```
public Nutrida(String nombre, int edad, double peso) {  
    // asignaciones  
    amistades = new Amistad[ /* tamaño máximo */ ];  
  
    // actualizar contador estático  
}
```

Constructor 2 (con copia de array)

Estructura que debes rellenar:

```
public Nutrida(String nombre, int edad, double peso, Amistad[]
amistades) {
    this(nombre, edad, peso);

    this.amistades = new Amistad[ /* tamaño adecuado */ ];

    for (int i = 0; i < /* ??? */; i++) {
        this.amistades[i] = amistad[i]; // copia real
    }
}
```

Método esJuvenil

Solo la forma:

```
public boolean esJuvenil() {
    return /* comparar edad */;
}
```

Método tieneAmistadCon

Estructura típica a completar:

```
public boolean tieneAmistadCon(Nutrida otra) {
    for (int i = 0; i < amistad.length; i++) {
        if (amistad[i] != null && /* condición adecuada */) {
            return true;
        }
    }
    return false;
}
```

Método contarAmistadesIntensas

Pista clara:

```
int contador = 0;

for (int i = 0; i < amistades.length; i++) {
    if (amistades[i] != null && amistades[i]./* ??? */) {
        contador++;
    }
}

return contador;
```

Método privado contarAmistadesReales

Solo estructura:

```
private int contarAmistadesReales() {
    int cuenta = 0;

    for (int i = 0; i < amistades.length; i++) {
        if (amistades[i] != null) {
            cuenta++;
        }
    }

    return cuenta;
}
```

3. CLASE Colonia

Atributos

Ya conocidos:

```
private String nombre;
private Nutrida[] integrantes;
```

Campos estáticos

Solo firma:

```
public static final int CAPACIDAD_MAXIMA = /* ??? */;  
public static int coloniasCreadas = /* ??? */;
```

Constructores

1. Con array:

```
public Colonia(String nombre, Nutrida[] integrantes) {  
    // asignaciones  
    // incrementar contador  
}
```

2. Vacía:

```
public Colonia(String nombre) {  
    // asignar nombre  
    // crear array vacío  
    // incrementar contador  
}
```

Método pesoMedio

Pista del cálculo:

```
double suma = 0;  
  
for (int i = 0; i < integrantes.length; i++) {  
    if (integrantes[i] != null) {  
        suma += /* peso */;  
    }  
}  
  
return suma / /* número de nutridas reales */;
```

Método buscarPorNombre

Estructura:

```
for (int i = 0; i < integrantes.length; i++) {
    if (integrantes[i] != null &&
        integrantes[i].getNombre().equals(/* ??? */)) {

        return integrantes[i];
    }
}
return null;
```

Método contarEnlacesAmistad

Plantilla:

```
int total = 0;

for (int i = 0; i < integrantes.length; i++) {
    if (integrantes[i] != null) {
        Amistad[] arr = integrantes[i].getAmistades();

        for (int j = 0; j < arr.length; j++) {
            if (arr[j] != null) {
                total++;
            }
        }
    }
}

return total;
```

Método privado contarNutridasReales

```
private int contarNutridasReales() {
    int cuenta = 0;

    for (int i = 0; i < integrantes.length; i++) {
        if (integrantes[i] != null) {
            cuenta++;
        }
    }

    return cuenta;
}
```

4. CLASE GestorColonias

Método crearColoniaFiltrada

Estructura:

```
public static Colonia crearColoniaFiltrada(Colonia origen, double
pesoMinimo) {
    // contar cuántas nutridas cumplen
    // crear array del tamaño exacto
    // copiar nutridas que cumplen
    // devolver nueva colonia
}
```

Método sonCompatibles

Guía de posibles pasos:

```
boolean edadOK = /* diferencia <= 5 */;
boolean pesoOK = /* peso conjunto <= 25 */;

// contar amistades reales de cada nutrida
int amistadesA = 0;
int amistadesB = 0;

// recorrer amistades de A
// recorrer amistades de B

boolean amistadesOK = (amistadesA < 5 && amistadesB < 5);

return edadOK && pesoOK && amistadesOK;
```

5. MAIN demostrativo

Pistas:

1. Primero crea nutridas **sin amistades**.
2. Luego crea amistades.
3. Luego crea nutridas **con** amistades.
4. Crea colonias.
5. Usa métodos de Colonia.
6. Usa métodos de GestorColonias.

Estructura típica:

```
public static void main(String[] args) {  
  
    // 1. nutridas sin amistades  
  
    // 2. amistades apuntando a nutridas previas  
  
    // 3. nutridas con array de amistades  
  
    // 4. colonias  
  
    // 5. métodos de colonia  
  
    // 6. métodos de gestor  
}
```


6. CHECKLIST final

Antes de entregar, debes verificar:

- ¿El constructor con amistades copia el array?
- ¿El método privado se usa indirectamente (no se llama desde fuera)?
- ¿Los contadores estáticos aumentan en los constructores?
- ¿Han comprobado nulls en arrays?
- ¿Las intensidades se validan correctamente?
- ¿Se usan condiciones compuestas con && donde toca?
- ¿Cada clase hace solo lo que le corresponde?